

PART OF A SERIES OF TUTORIALS ON QUANTUM ERROR CORRECTION

Rotated Surface Codes

From local checks to lattice surgery

Revised 23 September 2026

In the previous tutorial, we introduced stabilizers, error syndromes, and decoding, and began to examine faults in syndrome-measurement circuits. We now ask how to arrange those circuits on a device. On many platforms, including superconducting-qubit arrays, direct two-qubit gates are restricted to nearby qubits. A useful code must respect that connectivity as it grows.

We will build a surface-code patch from local checks, understand errors through its geometry, and then use joint measurements between encoded qubits to implement a logical CNOT.

1 Locality matters

A **geometrically local code** arranges qubits so that each check acts on a small neighborhood, even as the code grows. The surface code is one example: its checks fit on a two-dimensional lattice.

The surface code arranges data and check qubits on a two-dimensional lattice. Its short-range interactions suit hardware with local connectivity, and its geometry makes checks, error paths, and logical operators visible. Experiments also show that, under suitable conditions, logical memory errors can decrease as the code grows [6].

2 The rotated surface code

The foundational constructions are Kitaev's toric code, first circulated in 1997, and Bravyi and Kitaev's planar codes with boundaries in 1998 [1, 2]; repeated noisy correction was developed further by Dennis et al. [3]. The conventional surface-code construction places data qubits on **edges**. An X check acts on edges meeting at a vertex; a Z check acts around a face. Figure 1 uses orange and teal support shading to make these check neighborhoods explicit.

The rotated patch chooses boundaries at 45° to the underlying lattice and trims boundary qubits. Redrawn as a square grid, its data qubits sit at the **corners of checkerboard faces** [4, 7]. Both layouts are planar surface-code encodings, but their finite patches have different boundaries, check supports, and qubit counts. The rotated cut reduces the number of data qubits while preserving the intended code distance.

Throughout this tutorial, we draw X checks in orange and Z checks in teal. In a common implementation with one measurement ancilla per check, each colored region requires one ancilla near its center. A rotated patch then uses $d^2 - 1$ check ancillas in addition to its d^2 data qubits. Reusing ancillas can change this hardware count.

2.1 Reading the checks and boundaries

In the rotated patch shown here, each colored face is also called a **plaquette**. A bulk plaquette contains four data-qubit corners; a boundary half-plaquette contains two. The corresponding check multiplies the same Pauli operator over those corners:

$$S_X(f) = \prod_{j \in \text{corners}(f)} X_j, \quad S_Z(f) = \prod_{j \in \text{corners}(f)} Z_j.$$

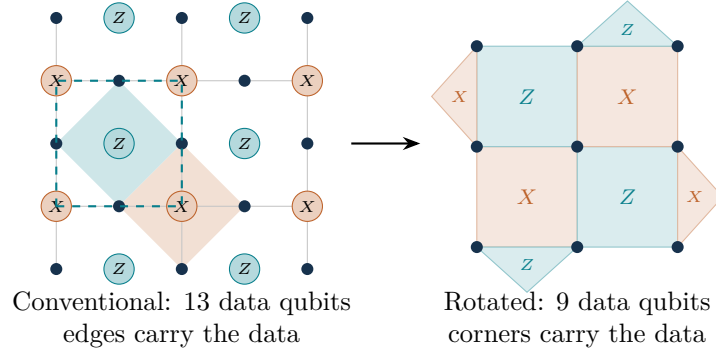


Figure 1: Two distance-three patches. Black dots are data qubits. The dashed square highlights a conventional Z face; its check acts on the four data qubits around its perimeter. In the rotated layout, each colored region acts on its data-qubit corners.

In Figure 2, the numbers run left to right along each row, beginning with 0–4 at the bottom, then 5–9 above. These labels let us name the physical operators in an example.

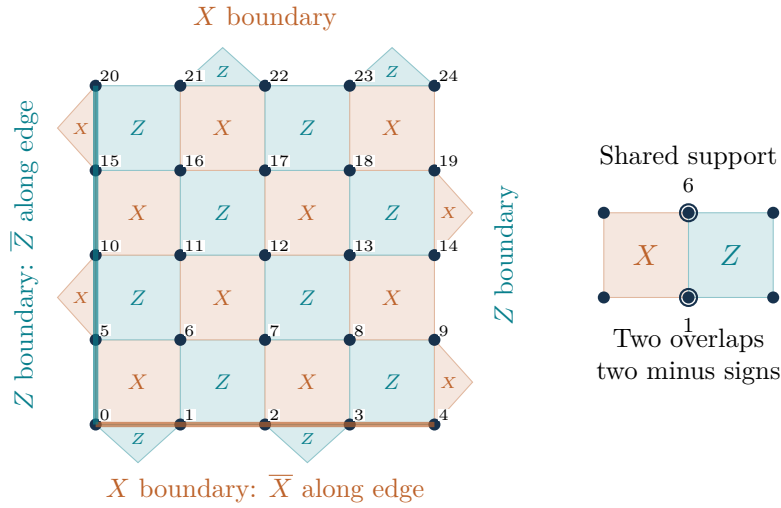


Figure 2: A distance-five patch: 25 data qubits and 24 checks. The inset highlights shared qubits 1 and 6. Boundary labels name logical operators supported *along* an edge; the paths represent $\bar{X}$ and $\bar{Z}$, explained in the logical-operator subsection.

The bottom-left orange plaquette and its teal neighbor give

$$S_X = X_0 X_1 X_5 X_6, \quad S_Z = Z_1 Z_2 Z_6 Z_7.$$

The bottom teal half-plaquette gives $Z_0 Z_1$. Every X – Z check pair overlaps on zero or two qubits, including at boundaries. Like-type checks commute automatically. Therefore, all stabilizer checks can be imposed simultaneously.

We assign each data vertex a **local Pauli label**¹ by multiplying the Pauli types of all stabilizers incident on that vertex, while ignoring overall phase. If $n_X(q)$ and $n_Z(q)$ are the numbers of incident X and Z stabilizers at vertex q , then

$$\chi(q) = X^{n_X(q) \bmod 2} Z^{n_Z(q) \bmod 2}.$$

Thus two X labels cancel, two Z labels cancel, and one X together with one Z gives Y , up to phase. For example, in Figure 2,

$$\chi(0) = XZ \sim Y, \quad \chi(1) = XZZ \sim X, \quad \chi(6) = XXZZ = I.$$

¹In Entropica’s visual language, $\chi(q)$ may be called a **local Pauli charge**.

These labels depend on the checks as drawn; they are a visual aid, not additional measured checks. They make the boundary pattern visible: along one pair of edges their product gives X , and along the perpendicular pair it gives Z . The stabilizer layout mathematically determines the boundaries; the local labels provide a quick way to see them. In the following subsection, we use these boundary types to identify the logical X and Z operators.

2.2 Error strings, boundaries, and logical operators

A Pauli error changes the sign of every check with which it anticommutes. A **string** is a product of same-type Pauli errors along a path. Therefore, a check is violated exactly when it overlaps that string on an odd number of qubits. Thus, checks in the middle of a string usually overlap it twice and remain unchanged, while checks at its endpoints overlap it once and return -1 . The syndrome therefore marks the string's visible endpoints.

One absorbed endpoint. Consider the bottom-left data qubit 0. A Z_0 error anticommutes with the neighbouring X check $X_0X_1X_5X_6$, so that check returns -1 . The boundary check Z_0Z_1 commutes with Z_0 , and there is no second X check below the patch. Thus Z_0 has one visible endpoint and one endpoint absorbed at the bottom boundary.

In this convention, a Z string creates defects on X checks and may end at a top or bottom X boundary. An X string creates defects on Z checks and may end at a left or right Z boundary. At a corner the two boundary types meet. A $Y = iXZ$ error has both components; the corner is not a third, “ Y -boundary.”

Two absorbed endpoints: logical operators. The bottom-edge representative

$$\overline{X} = X_0X_1X_2X_3X_4$$

illustrates the cancellation directly. The leftmost boundary Z check sees X_0X_1 , the next bulk Z check sees X_1X_2 , and so on. Every relevant Z check therefore has two anticommuting overlaps and returns $+1$. The two geometric endpoints lie on the left and right Z boundaries, where the remaining endpoint defects are absorbed. Consequently, $\overline{X}$ has the trivial syndrome.

However, a trivial syndrome does not imply that the logical state is unchanged. This string connects the two distinct Z boundaries of the patch. No product of local stabilizer checks can remove such a boundary-to-boundary path or deform it into a small region. It is therefore not equivalent to the identity on the code space. Any X string joining these two Z boundaries can be deformed into this representative by multiplying by stabilizer checks, so these strings belong to the same logical class.

The same reasoning gives the vertical representative

$$\overline{Z} = Z_0Z_5Z_{10}Z_{15}Z_{20}.$$

The edge label and the absorbing boundary should not be confused. We draw $\overline{X}$ *along* the bottom X -labelled edge, but its endpoints are absorbed at the left and right Z boundaries. Similarly, $\overline{Z}$ is drawn along the left Z -labelled edge while its endpoints are absorbed at the bottom and top X boundaries.

The two representatives intersect once, at qubit 0, so

$$\overline{X}\overline{Z} = -\overline{Z}\overline{X}.$$

We choose $\overline{Z}$ as the logical Z operator and define $|0_L\rangle$ and $|1_L\rangle$ as its $+1$ and -1 eigenstates. The anticommutation means that $\overline{X}$ flips this logical Z eigenvalue, so we call it the logical X operator. After fixing the relative phase of the logical basis states,

$$\overline{X}|0_L\rangle = |1_L\rangle, \quad \overline{X}|1_L\rangle = |0_L\rangle.$$

Different boundary types and mixed errors. A string that reaches an incompatible boundary remains detectable. For example, a Z string can begin at the bottom X boundary but end at a left Z boundary. The bottom endpoint is absorbed, whereas the left endpoint leaves a violated X check. It has one visible e defect and is not logical. The converse holds for an X string from a left or right Z boundary to a top or bottom X boundary.

Mixed Pauli errors should be drawn by separating their X and Z components. Figure 3 shows $E = X_5 Z_2$. The X_5 factor violates the Z check $Z_5 Z_6 Z_{10} Z_{11}$, while the Z_2 factor violates the X check $X_2 X_3 X_7 X_8$. The error therefore produces one m defect and one e defect at different locations. These are not the two ends of one same-type string; they are the visible endpoints of two Pauli components.

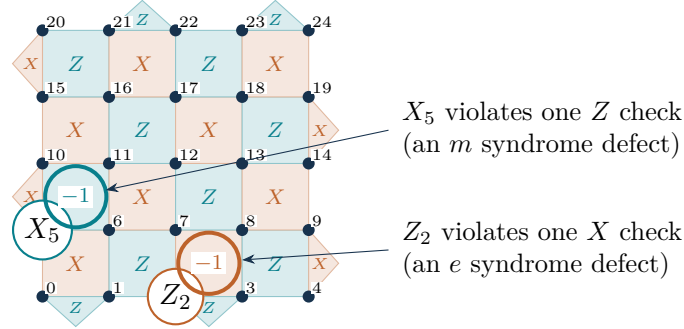


Figure 3: The mixed error $E = X_5 Z_2$. The X component violates one teal Z check, producing an m defect; the Z component violates one orange X check, producing an e defect. A mixed Pauli error is drawn as its separate X and Z components.

Zooming out with local Pauli labels. The local Pauli labels introduced above are useful because they make the boundary pattern visible before we draw every individual check. The square-patch picture in Figure 4, following the patch view in *A Game of Surface Codes* [5], keeps exactly the information needed here: the X and Z boundary types, visible syndrome defects, and the way a string connects boundaries. It hides the microscopic checks and must therefore be read using the cancellation rule established above. The local labels are a visual bookkeeping device; they are not the measured syndrome defects. The terms *rough* and *smooth* are also used for boundaries; with the e/m convention here, rough absorbs e and smooth absorbs m [3].

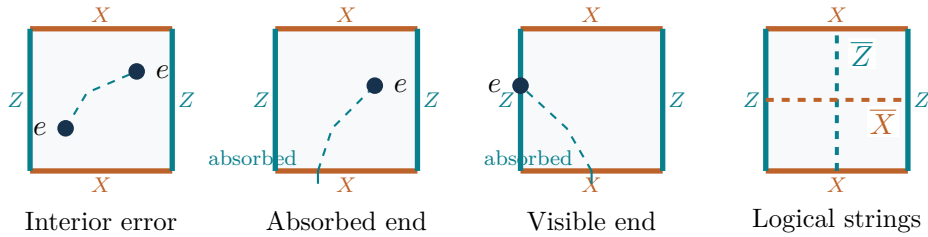


Figure 4: Zoomed-out string classes. A Z string has two visible e defects in the interior, one defect after reaching a compatible X boundary, or one defect after reaching an incompatible Z boundary. Logical strings join distinct compatible boundaries and have no visible endpoints.

2.3 Measuring a plaquette: gate order

The preceding tutorial derived ancilla check measurements. An X check uses an ancilla prepared in $|+\rangle$, acting as the CNOT control, and measured in X . A Z check uses an ancilla prepared in $|0\rangle$, acting as the CNOT target, and measured in Z . Within an isolated, fault-free check, the

order of these CNOTs does not affect the measurement. However, faults can occur during the measurement, and the order determines how they spread to the data.

For a CNOT with control c and target t , write $U = \text{CNOT}_{c \rightarrow t}$. Pauli propagation obeys

$$\begin{aligned} UX_cU^\dagger &= X_cX_t, & UX_tU^\dagger &= X_t, \\ UZ_tU^\dagger &= Z_cZ_t, & UZ_cU^\dagger &= Z_c. \end{aligned} \quad (1)$$

Thus an X error spreads from control to target, while a Z error spreads from target to control. In the opposite directions, these errors do not spread.

For an X check, an ancilla X fault can therefore propagate to several data qubits through the remaining CNOTs; an ancilla Z fault does not spread to the data, although it can flip the ancilla's X readout. For a Z check, the roles reverse: an ancilla Z fault can spread to the data, while an ancilla X fault can flip the Z readout. A single circuit fault that spreads through the ancilla to produce a correlated data error is called a *hook error*. We therefore orient X -check hooks to limit their contribution to a logical X error, and Z -check hooks to limit their contribution to a logical Z error.

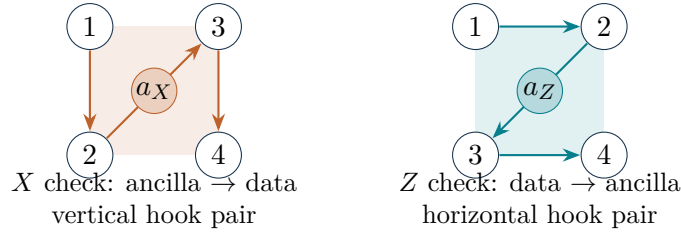


Figure 5: CNOT orders for the patch orientation used here. Corner numbers give the time steps at which the ancilla interacts with each data qubit; they are not qubit labels. The N-shaped X order makes the final pair vertical, perpendicular to a shortest logical X string. The Z-shaped Z order makes the final pair horizontal, perpendicular to a shortest logical Z string.

The key case is an ancilla fault between the second and third CNOTs²: it spreads to the last two data qubits. In the N-shaped X order, these form a vertical pair. Since our shortest logical X strings run horizontally, this hook does not supply two neighboring sites along such a string. Similarly, the Z-shaped Z order produces a horizontal pair, across rather than along a shortest vertical logical Z string [7]. Note that these choices prevent the two-qubit hooks from providing a shortcut along the corresponding logical direction; they do not eliminate all ways a logical error can occur.

Can neighboring checks run together? Two neighboring checks share data qubits, so they cannot both interact with the same qubit at the same instant. They can nevertheless be measured during the same four-step interval: at each step, their ancillas interact with *different* data qubits. This is what we mean by running the checks in parallel.

Consider the neighboring checks

$$S_X = X_0X_1X_5X_6, \quad S_Z = Z_1Z_2Z_6Z_7.$$

They share qubits 6 and 1. Following the N-shaped and Z-shaped orders, their CNOTs are interleaved as shown below.

²Try placing the propagating ancilla fault before, between, and after the CNOTs. Track which data qubits it reaches, then simplify the resulting error by multiplying by the measured stabilizer. A four-qubit error is the stabilizer itself, a three-qubit error is equivalent to a single-qubit error, and the two-qubit case explains the orientation choice.

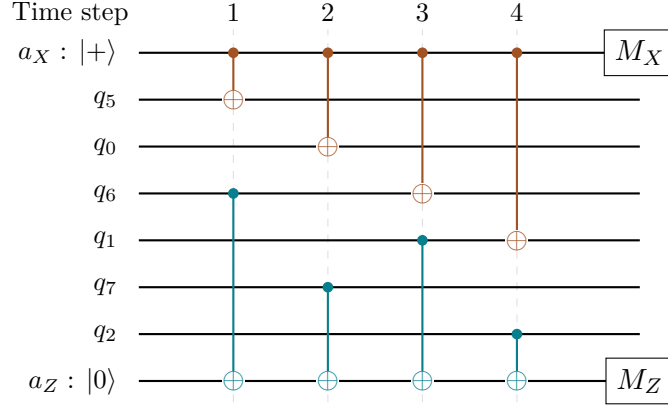


Figure 6: Two neighboring check measurements in parallel. Time runs from left to right; gates in the same numbered column occur simultaneously. A filled dot marks a CNOT control and $\oplus$ its target. Orange gates belong to the X check and teal gates to the Z check. A vertical line crossing another wire does not act on that wire unless a gate symbol is shown.

At each time step, the two ancillas interact with distinct data qubits, so there are no gate collisions. Their order must also preserve the intended parity measurements.

For example, suppose the shared qubits q_6 and q_1 start at 0,0. In the $|1\rangle$ component of the X ancilla, its CNOTs flip both bits. If the Z ancilla interacts before both flips, it encounters 0,0; after both flips, it encounters 1,1. Both have even parity. But interacting before one flip and after the other gives 0,1 or 1,0, changing the parity.

Thus the relative order must agree at both shared qubits. Our schedule puts the Z interaction first at both, so the parallel measurements preserve the intended checks.

3 Lattice surgery: a joint logical measurement

To compute with stored information, we need interactions between logical patches. **Lattice surgery** temporarily changes the local checks where patches meet. Their outcomes give a joint logical measurement without revealing either logical value separately [4]. We work through a $\overline{Z}_L \overline{Z}_R$ measurement, then use joint measurements to build a CNOT. The complementary Jupyter notebook explores other operations, including $\overline{Z}_L \overline{X}_R$.

3.1 Measuring joint parity with local checks

We want to measure $P = \overline{Z}_L \overline{Z}_R$, the logical Z parity check between two distance-three patches with data qubits $L_0, \dots, L_8$ and $R_0, \dots, R_8$, numbered by rows as before. Their facing edges support

$$\overline{Z}_L = Z_{L_2} Z_{L_5} Z_{L_8}, \quad \overline{Z}_R = Z_{R_0} Z_{R_3} Z_{R_6}.$$

Place three additional *bridge data qubits* b_0, b_1, b_2 , each prepared in $|+\rangle$, between the patches. These are data qubits used to join the patches; separate ancillas measure the new checks.

Measure the four new seam checks

$$\begin{aligned} G_1 &= Z_{L_2} Z_{b_0}, & G_2 &= Z_{b_0} Z_{b_1} Z_{R_0} Z_{R_3}, \\ G_3 &= Z_{L_5} Z_{L_8} Z_{b_1} Z_{b_2}, & G_4 &= Z_{b_2} Z_{R_6}. \end{aligned} \tag{2}$$

In the product of the four checks, each bridge factor occurs twice and cancels because $Z_{b_i}^2 = I$. Thus

$$G_1 G_2 G_3 G_4 = \overline{Z}_L \overline{Z}_R, \quad m = g_1 \oplus g_2 \oplus g_3 \oplus g_4, \tag{3}$$

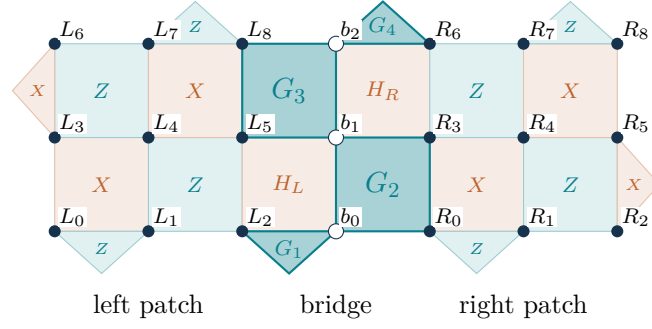


Figure 7: An explicit ZZ merge. Hollow dots are bridge data qubits, ordered bottom to top. Four highlighted Z checks $G_1, \dots, G_4$ span the seam. The other faces show the compatible checks of the merged rectangular patch. Each region is measured with a check ancilla, omitted here.

Here $g_j = 0$ denotes a $+1$ result for G_j , and $g_j = 1$ denotes -1 . The joint outcome bit is m : its measured sign is $(-1)^m$. Thus $m = 0$ means the two logical Z values agree, while $m = 1$ means they differ. The result $m = 0$ alone does not distinguish $|00\rangle_L$ from $|11\rangle_L$.

Adding the seam Z checks requires us to replace two old X checks at the facing boundaries. For example, the old left check $B_L = X_{L_2}X_{L_5}$ anticommutes with G_1 because they overlap only at L_2 . Suppose a seam measurement gives $G_1|\psi\rangle = |\psi\rangle$. If we then measure B_L and obtain $+1$, the state is proportional to $(I + B_L)|\psi\rangle$. But

$$G_1|\psi\rangle = |\psi\rangle, \quad G_1B_L|\psi\rangle = -B_L|\psi\rangle.$$

The two terms have opposite G_1 signs. Measuring G_1 again gives either sign with equal probability, even though no data error occurred. The old check B_L therefore cannot remain a check of the merged patch. The old right check $X_{R_3}X_{R_6}$ similarly conflicts with G_2 .

We replace them with

$$H_L = X_{L_2}X_{L_5}X_{b_0}X_{b_1}, \quad H_R = X_{b_1}X_{b_2}X_{R_3}X_{R_6}.$$

Each replacement has an even number of overlaps with every seam Z check, so they commute. Both start with sign $+1$, because they are products of an old X check and X operators on bridge qubits prepared in $|+\rangle$. Together with the retained checks, they define the merged patch.

3.2 Splitting and accounting for the outcomes

The merge has measured the joint parity. To separate the patches again, measure the three bridge qubits in X and restore the old boundary checks. Let $r_i = 0$ mean an X_{b_i} result of $+1$, and $r_i = 1$ mean -1 .

Before the split, the replacement checks H_L and H_R have sign $+1$. Using their definitions, the restored boundary checks must therefore have signs

$$X_{L_2}X_{L_5} : (-1)^{r_0 \oplus r_1}, \quad X_{R_3}X_{R_6} : (-1)^{r_1 \oplus r_2}.$$

For example, if $r_0 \oplus r_1 = 1$, the left boundary check has sign -1 in order to have $H_L + 1$. This is a known outcome of the split, not an unknown data error.

The bridge measurements also affect the signs between terms in the data state. Consider the middle bridge qubit b_1 . Its X measurement gives either $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ or $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$. Compared with a $|+\rangle$ result, a $|-\rangle$ result puts a minus sign on the part of the state paired with $b_1 = 1$.

The seam checks tell us how that sign appears on the data:

$$G_1 G_2 = Z_{L_2} Z_{b_1} Z_{R_0} Z_{R_3}.$$

Once the outcomes of G_1 and G_2 are known, the extra minus signs from a $|-\rangle$ result on b_1 have the same effect on the data as $Z_{L_2} Z_{R_0} Z_{R_3}$, apart from an overall sign. We can write this data operator as

$$Z_{L_2} Z_{R_0} Z_{R_3} = Z_{L_2} Z_{R_6} \bar{Z}_R.$$

The first two factors affect local boundary-check signs, while the last factor is a logical Z on the right logical qubit. Accounting for all three bridge outcomes gives

$$D_{\mathbf{r}} = Z_{L_2}^{r_0 \oplus r_1} Z_{R_6}^{r_1 \oplus r_2} \bar{Z}_R^{r_1}. \quad (4)$$

The first two factors account for the restored boundary-check signs; the last records the logical Z effect. We keep track of $D_{\mathbf{r}}$ in a **Pauli frame**: a classical record used to interpret later measurements, without applying extra physical gates.

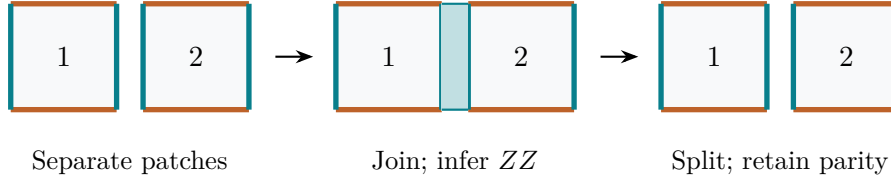


Figure 8: Join the patches to measure their joint parity, then measure the bridge qubits to separate them. Their outcomes determine the Pauli frame in Eq. (4).

This description assumes ideal measurements. With noisy measurements, the seam checks must be repeated and decoded before their joint parity can be trusted. Exchanging X and Z gives a joint XX measurement on facing X edges. At the logical level, a $\bar{Z}_L \bar{X}_R$ measurement can be understood by applying a Hadamard basis change to the right patch, measuring $\bar{Z}_L \bar{Z}_R$, and undoing the basis change. Implementing that basis change is an additional operation, not part of the ZZ seam shown here.

We now use ZZ and XX measurements to build a CNOT.

4 A CNOT using an ancillary logical patch

Let C and T be arbitrary control and target logical qubits. Prepare a third logical patch A in $|+\rangle_L$. In this section, X_C, Z_C , and analogous symbols denote *logical* Paulis; we omit overbars to keep the derivation readable. The table below shows the procedure for applying a CNOT between two logical patches.

Step	Measurement	Result bit
1	$Z_C Z_A$	a
2	$X_A X_T$	b
3	Z_A	c

For each result bit, 0 means a measured sign of $+1$ and 1 means -1 . Each split updates the Pauli frame used to interpret later results. The bits a, b, c denote logical outcomes interpreted using the current frame. Patch A is then no longer needed for this CNOT and can be reset for later use.

The three results tell us which known Pauli correction accompanies the CNOT:

$$F = Z_C^b X_T^{a \oplus c}.$$

For example, if $a = 1$ and $b = c = 0$, then $F = X_T$: the target has a known extra X . We can correct it physically, or record F in the **Pauli frame** and use that record to interpret later measurements.

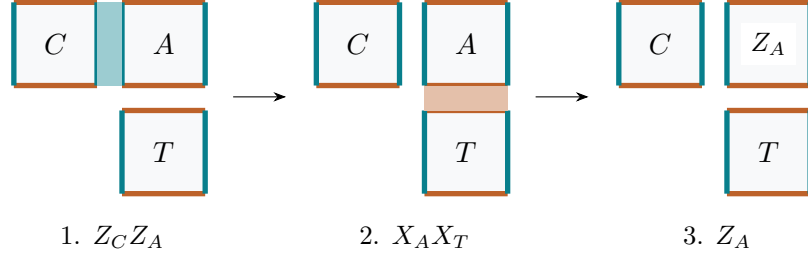


Figure 9: An L-shaped layout: C and A meet on Z edges; A and T meet on X edges. The highlighted seams are used successively, with the patches split again after each parity operation. The last panel measures the ancillary logical patch.

4.1 Checking the CNOT by tracking logical operators

We can verify the CNOT without following every possible input state. Instead, we track the four logical Pauli operators X_C, Z_C, X_T, Z_T through the measurements. For an ideal CNOT with control C and target T , the expected rules are

$$X_C \mapsto X_C X_T, \quad Z_C \mapsto Z_C, \quad X_T \mapsto X_T, \quad Z_T \mapsto Z_C Z_T.$$

We will show that our measurement procedure gives these rules, apart from signs determined by its known outcomes.

Start with X_C . The ancilla was prepared in $|+\rangle_L$, so initially $X_A = +1$ and X_C acts like $X_C X_A$. This operator commutes with both joint measurements: it has two anticommuting overlaps with $Z_C Z_A$ and commutes with $X_A X_T$. After the second measurement, $X_A X_T = (-1)^b$, so X_A acts like $(-1)^b X_T$. Thus X_C reaches the output as

$$X_C \mapsto (-1)^b X_C X_T.$$

Now consider Z_T . After the first measurement, $Z_C Z_A = (-1)^a$, so Z_T acts like $(-1)^a Z_C Z_A Z_T$. This operator commutes with $X_A X_T$: it anticommutes on both A and T , giving two minus signs that cancel. The final measurement gives $Z_A = (-1)^c$. Hence

$$Z_T \mapsto (-1)^{a \oplus c} Z_C Z_T.$$

The other two operators, Z_C and X_T , commute with all three measurements and remain unchanged. The four rules are therefore

$$\begin{aligned} X_C &\mapsto (-1)^b X_C X_T, & Z_C &\mapsto Z_C, \\ X_T &\mapsto X_T, & Z_T &\mapsto (-1)^{a \oplus c} Z_C Z_T. \end{aligned}$$

A CNOT has these same rules without the two outcome-dependent signs. These four Pauli rules determine the two-qubit operation up to an overall scalar. Both signs are accounted for by the known Pauli $F = Z_C^b X_T^{a \oplus c}$. For example, Z_C changes the sign of X_C when $b = 1$, while X_T

changes the sign of Z_T when $a \oplus c = 1$. Recording F in the Pauli frame therefore leaves exactly the logical action of a CNOT, for any input state.

Each measurement has two equally likely outcomes: $Z_C Z_A$ is unbiased because A starts in $|+\rangle_L$; $X_A X_T$ is unbiased after the first measurement because it anticommutes with $Z_C Z_A$; and Z_A is unbiased after the second because it anticommutes with $X_A X_T$. Each branch thus has probability $1/8$. If K_{abc} denotes its unnormalized operation, we have

$$K_{abc} \doteq \frac{1}{2\sqrt{2}} Z_C^b X_T^{a \oplus c} \text{CNOT}_{C \rightarrow T}, \quad (5)$$

where $\doteq$ means equality up to an overall phase, which has no effect on the state.

We have moved from local checks to a logical CNOT: seam checks reveal joint parity, and the measurement outcomes determine the Pauli frame. This ideal operator argument does not establish fault tolerance. The next tutorial examines noisy measurements, repeated rounds, and decoding.

References

- [1] A. Kitaev, *Fault-tolerant quantum computation by anyons*, Ann. Phys. 303, 2–30 (2003); preprint 1997. [arXiv:quant-ph/9707021](#).
- [2] S. B. Bravyi and A. Yu. Kitaev, *Quantum codes on a lattice with boundary* (1998). [arXiv:quant-ph/9811052](#).
- [3] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, *Topological quantum memory*, J. Math. Phys. 43, 4452–4505 (2002). [arXiv:quant-ph/0110143](#).
- [4] C. Horsman, A. G. Fowler, S. Devitt, and R. Van Meter, *Surface code quantum computing by lattice surgery*, New J. Phys. 14, 123011 (2012). [arXiv:1111.4022](#).
- [5] D. Litinski, *A Game of Surface Codes: Large-Scale Quantum Computing with Lattice Surgery*, Quantum 3, 128 (2019). [doi:10.22331/q-2019-03-05-128](#).
- [6] Google Quantum AI and Collaborators, *Quantum error correction below the surface code threshold*, Nature 638, 920–926 (2025). [doi:10.1038/s41586-024-08449-y](#).
- [7] Y. Tomita and K. M. Svore, *Low-distance surface codes under realistic quantum noise*, Phys. Rev. A 90, 062320 (2014). [arXiv:1404.3747](#).